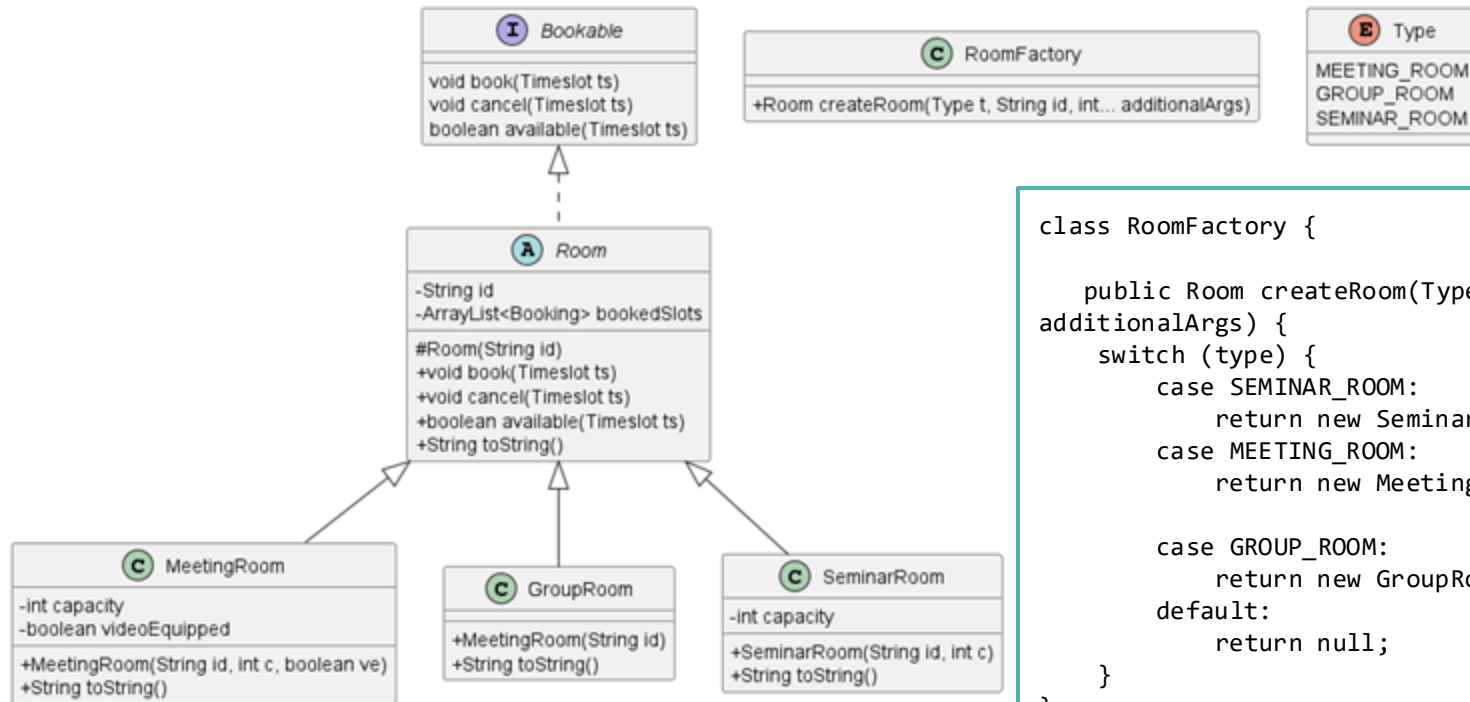

Design Patterns II

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

Simple Factory

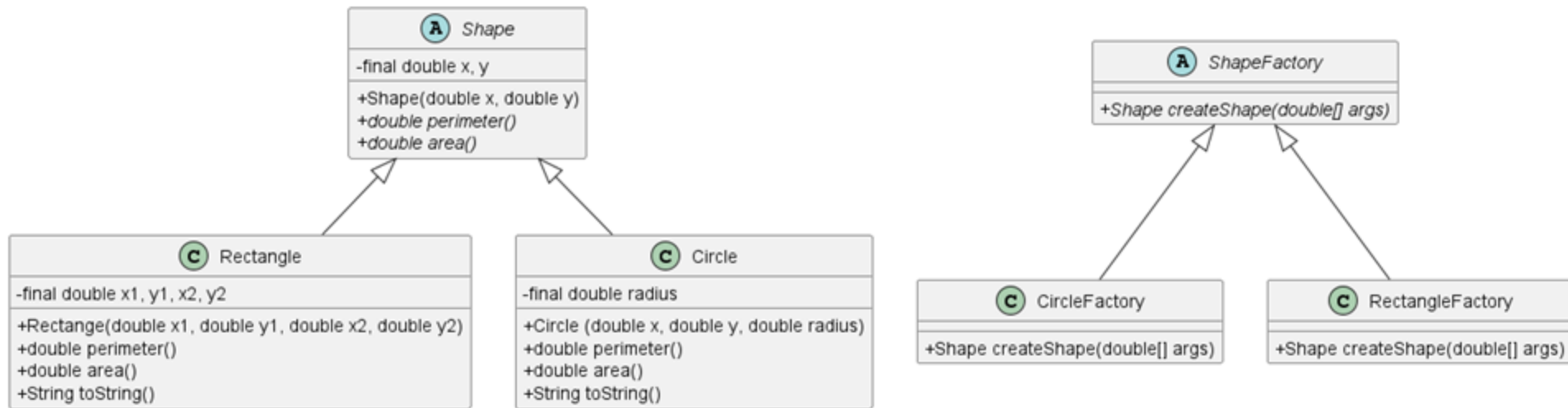


```

class RoomFactory {

    public Room createRoom(Type type, String id, int
additionalArgs) {
        switch (type) {
            case SEMINAR_ROOM:
                return new SeminarRoom(id, additionalArgs[0]);
            case MEETING_ROOM:
                return new MeetingRoom(id, additionalArgs[0],
                    (additionalArgs[1] != 0));
            case GROUP_ROOM:
                return new GroupRoom(id);
            default:
                return null;
        }
    }
}
    
```

Factory Method



```
class CircleFactory extends ShapeFactory {

    public Shape createShape(double[] args) {
        return new Circle(args[0], args[1], args[2]);
    }

}
```

```
class RectangleFactory extends ShapeFactory {

    public Shape createShape(double[] args) {
        return new Rectangle(args[0], args[1], args[2], args[3]);
    }

}
```

Creational Patterns

- Simple Factory
- Factory Method
- Abstract Factory
- Builder
- ...

Structural Patterns

- Decorator
- Adapter
- Composite
- ...

Behavioral Patterns

- Template Method
- Observer
- Command
- Strategy
- Visitor
- ...

Builder Pattern

- It separates the construction of a complex object from its representation.
- The same construction processes can create different representations.

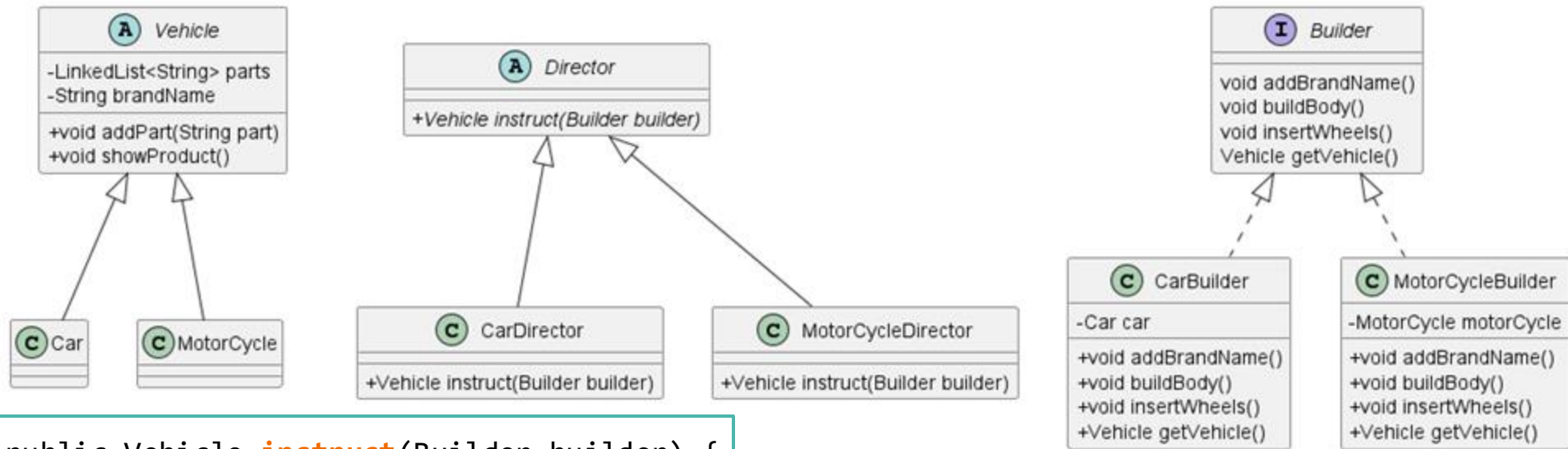
```
import java.util.Calendar;
...

Calendar c = new Calendar.Builder().setCalendarType("gregorian").setWeekDate(2023,
    45, Calendar.THURSDAY).build();
System.out.println(c);
Date d = c.getTime();
System.out.println(d);
```

Thu Nov 09 00:00:00 CET 2023

It would be 2nd day of the 11th month if Buddhist calendar is used

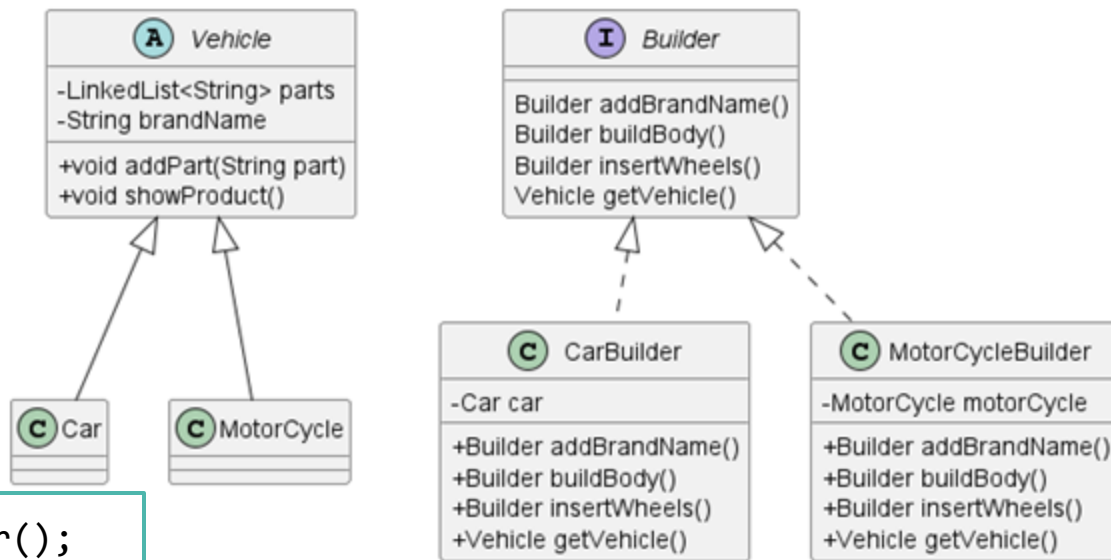
Builder that relies on a director, to build a vehicle.



```
public Vehicle instruct(Builder builder) {
    builder.buildBody();
    builder.insertWheels();
    builder.addBrandName();
    return builder.getVehicle();
}
```

```
Builder builder = new CarBuilder();
Director director = new CarDirector();
Vehicle vehicle = director.instruct(builder);
vehicle.showProduct();
```

The client builds the vehicle using the builder.



```
Builder builder = new CarBuilder();
Vehicle vehicle = builder.buildBody()
    .insertWheels()
    .addBrandName()
    .getVehicle();
vehicle.showProduct();
```

Because the return type is Builder, the methods can be chained.

Messy constructors versus a builder

```
// Case 1: Car without navigation system or climatisation
Car car1 = new Car("Red", "Sedan", 2024, false, null, false, null);

// Case 2: Car with navigation system, version specified, but without climatisation
Car car2 = new Car("Blue", "SUV", 2023, true, "v2.5", false, null);

// Case 3: Car with both navigation system and climatisation specified
Car car3 = new Car("Black", "Coupe", 2025, true, "v3.1", true, "Automatic");
```

```
// Case 1: Car without navigation system or climatisation
Car car1 = new Car.Builder("Red", "Sedan", 2024).setHasNavigationSystem(false).setHasClimatisation(false).build();

// Case 2: Car with navigation system and version specified, but without climatisation
Car car2 = new Car.Builder("Blue", "SUV", 2023).setHasNavigationSystem(true).setNavigationSystemVersion("v2.5")
    .setHasClimatisation(false).build();

// Case 3: Car with both navigation system and climatisation specified
Car car3 = new Car.Builder("Black", "Coupe", 2025).setHasNavigationSystem(true).setNavigationSystemVersion("v3.1")
    .setHasClimatisation(true).setClimatisationType("Automatic").build();
```


Use the Builder pattern to create courses

```
public class Course {  
    private String code;  
    private String title;  
  
    private Course() { }  
    public String getCode() {  
        return this.code;  
    }  
    public String getTitle() {  
        return this.title;  
    }  
    ...  
}
```

```
public class Course {  
    ...  
    public static class CourseBuilder {  
        private Course c;  
        public CourseBuilder() {  
            c = new Course();  
        }  
        public void setCode(String code); {  
            c.code = code;  
        }  
        public void setTitle(String title) {  
            c.title = title;  
        }  
        public Course getCourse() {  
            return c;  
        }  
    }  
}
```

```
Course.CourseBuilder cb = new Course.CourseBuilder();  
cb.setCode("OOP");  
cb.setTitle("Object-oriented programming");  
Course c = cb.getCourse();
```

Changes in Course do not affect the client code

```
public class Course {  
    private String code;  
    private String title;  
  
    private Course(String c, String t) {  
        code = c;  
        title = t;  
    }  
    public String getCode() {  
        return this.code;  
    }  
    public String getTitle() {  
        return this.title;  
    }  
    ...  
}
```

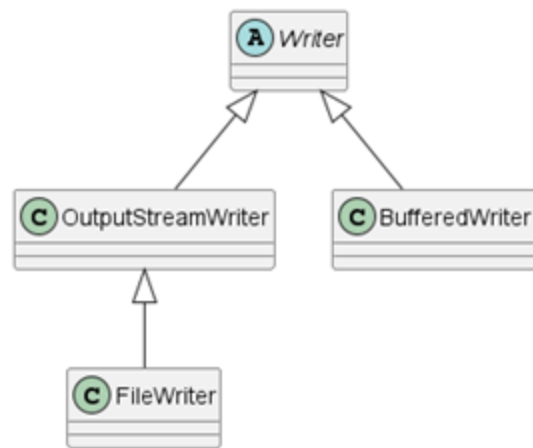
```
public class Course {  
    ...  
    public static class CourseBuilder {  
        private String ccode, ctitle;  
        public CourseBuilder() { }  
        public void setCode(String code) {  
            ccode = code;  
        }  
        public void setTitle(String title) {  
            ctitle = title;  
        }  
        public Course getCourse() {  
            return new Course(ccode, ctitle);  
        }  
    }  
}
```

```
Course.CourseBuilder cb = new Course.CourseBuilder();  
cb.setCode("OOP");  
cb.setTitle("Object-oriented programming");  
Course c = cb.getCourse();
```

Decorator Pattern

- It attaches additional responsibilities to an object dynamically.
- It relies on composition as a flexible alternative to subclassing.
- The decorator has the same interface as the decorated object (same interaction for the client).

```
FileWriter out = new FileWriter("courses.txt");  
BufferedWriter bw = new BufferedWriter(out);  
for (Course c : courses) {  
    bw.write(c.toString());  
}
```



Decorator for coffee, the base classes and interface

```
public interface Coffee {  
    String getDescription();  
    double getCost();  
}
```

```
public class SimpleCoffee implements Coffee {  
    @Override  
    public String getDescription() {  
        return "Simple coffee";  
    }  
  
    @Override  
    public double getCost() {  
        return 5.0; // Base cost of simple coffee  
    }  
}
```

```
public abstract class CoffeeDecorator implements  
Coffee {  
    protected Coffee coffee;  
  
    public CoffeeDecorator(Coffee coffee) {  
        this.coffee = coffee;  
    }  
  
    @Override  
    public String getDescription() {  
        return coffee.getDescription();  
    }  
  
    @Override  
    public double getCost() {  
        return coffee.getCost();  
    }  
}
```

Decorator for coffee, the decorator classes

```
public class MilkDecorator extends CoffeeDecorator
{
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", milk";
    }

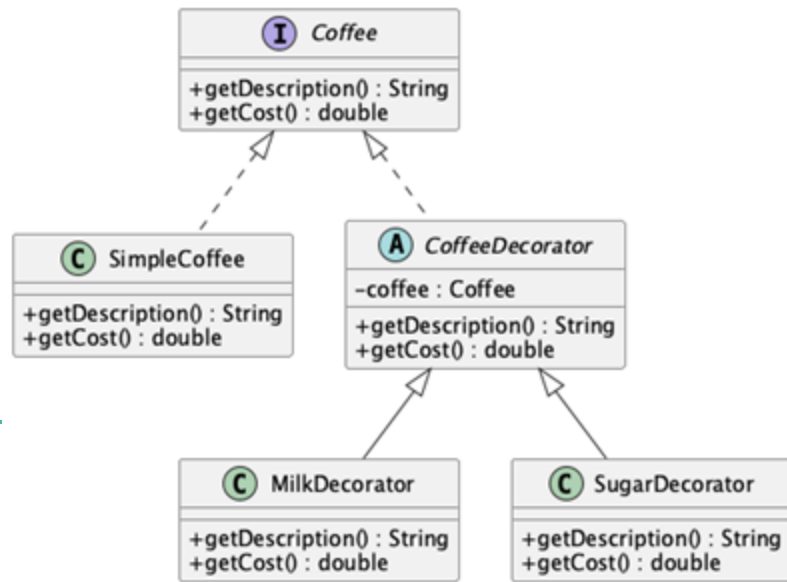
    @Override
    public double getCost() {
        return coffee.getCost() + 2;
        // Additional cost for milk
    }
}
```

```
public class SugarDecorator extends CoffeeDecorator
{
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", sugar";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 1;
        // Additional cost for sugar
    }
}
```

Decorator coffee, the client code



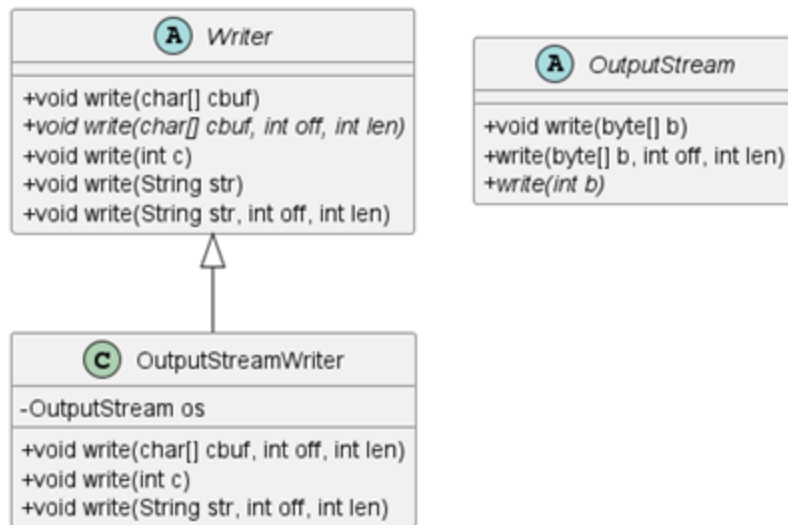
```
public class Main {
    public static void main(String[] args) {
        // Start with a simple coffee
        Coffee basicCoffee = new SimpleCoffee();
        System.out.println(basicCoffee.getDescription() + " - $" + basicCoffee.getCost());

        // Add milk to the coffee
        Coffee milkCoffee = new MilkDecorator(basicCoffee);
        System.out.println(milkCoffee.getDescription() + " - $" + milkCoffee.getCost());

        // Add sugar to the coffee with milk
        Coffee milkAndSugarCoffee = new SugarDecorator(milkCoffee);
        System.out.println(milkAndSugarCoffee.getDescription() + " - $" + milkAndSugarCoffee.getCost());
    }
}
```

Adapter Pattern

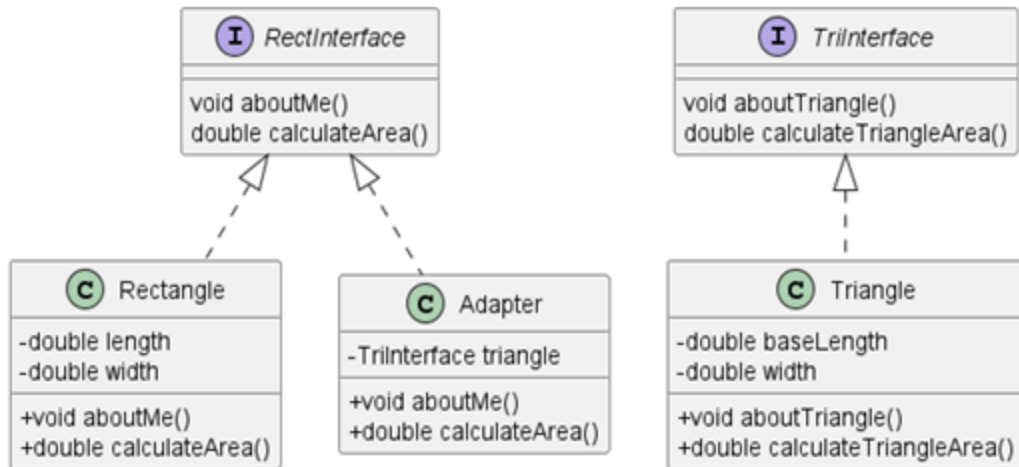
- Converts the *interface* of a class into another *interface* that the client expects.
- Client does not have to learn new ways to interface with the code.
- This pattern can be used to adapt classes and objects.



Object adapter

```
class Adapter implements RectInterface {  
    private TriInterface triangle;  
    ...  
    public double calculateArea() {  
        return triangle.calculateTriangleArea();  
    }  
}
```

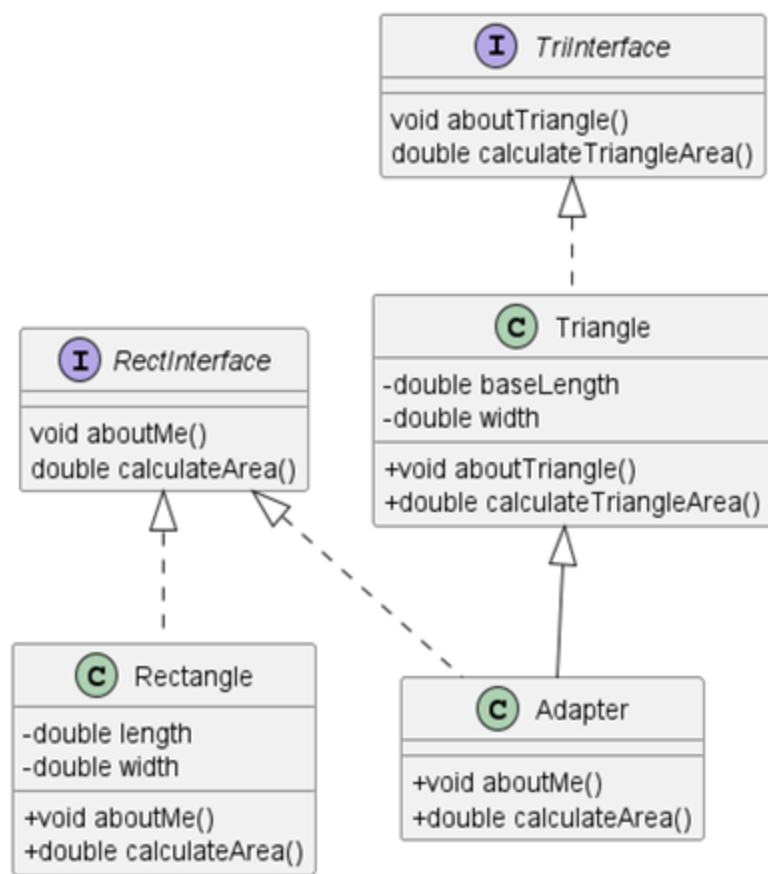
```
RectInterface rectangle = new Rectangle(20, 10);  
TriInterface triangle = new Triangle(20, 10);  
RectInterface adapter = new Adapter(triangle);  
List<RectInterface> rectangleObjects = new ArrayList<RectInterface>();  
rectangleObjects.add(rectangle);  
rectangleObjects.add(adapter);
```



Class adapter

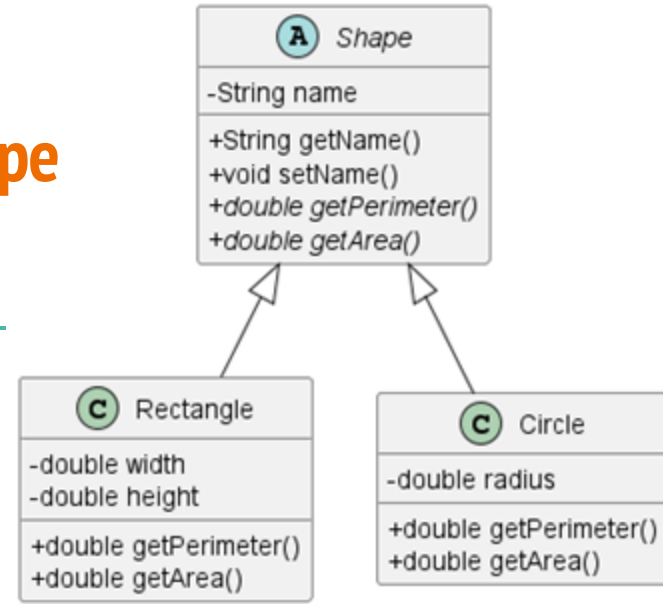
```
class Adapter extends Triangle implements RectInterface {  
    ...  
    public double calculateArea() {  
        return calculateTriangleArea();  
    }  
}
```

```
RectInterface rectangle = new Rectangle(20, 10);  
TriInterface triangle = new Triangle(20, 10);  
RectInterface adapter = new Adapter(20, 10);  
List<RectInterface> rectangleObjects = new ArrayList<RectInterface>();  
rectangleObjects.add(rectangle);  
rectangleObjects.add(adapter);
```



How can FigureCollection work with this Shape hierarchy?

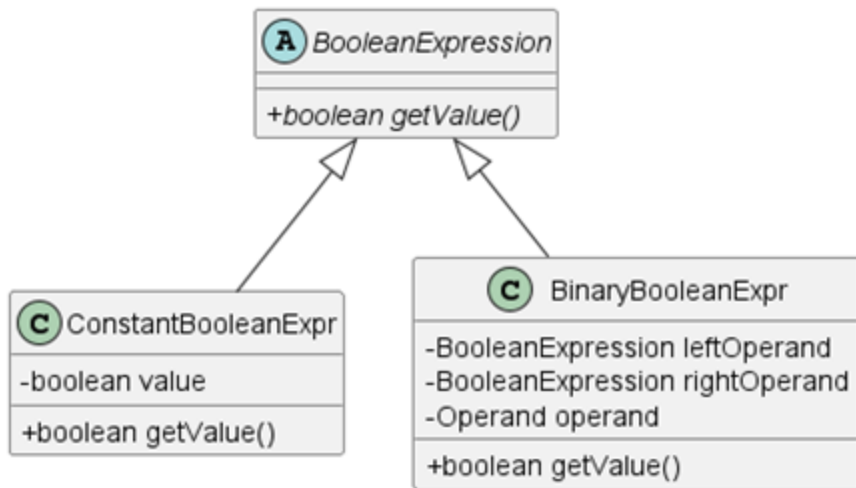
```
class FigureCollection {  
  
    private ArrayList<Figure> figures = new ArrayList<Figure>();  
    ...  
    public double getLargestArea() {  
        double largest = 0.0;  
        for (int i = 0; i < figures.size(); i++) {  
            Figure fi = figures.get(i);  
            double area = fi.area();  
            if (area > largest) {  
                largest = area;  
            }  
        }  
        return largest;  
    }  
}
```



1. No
2. Yes, but an adapter class is needed
3. Yes, but a decorator class is needed
4. Yes

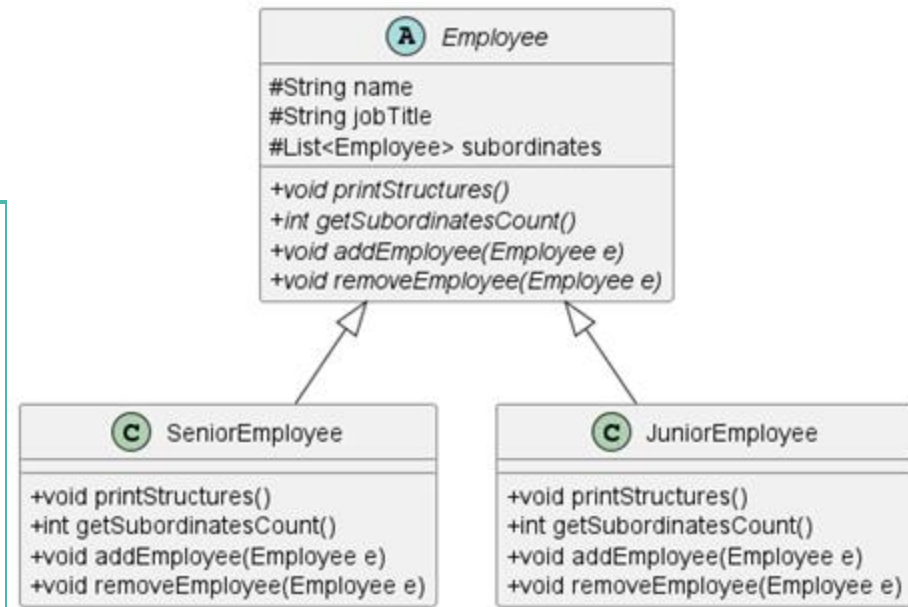
Composite Pattern

- Composes objects into tree structures to represent part-whole hierarchies.
- Individuals and compositions can be treated uniformly by the clients.



Composite Pattern, employees

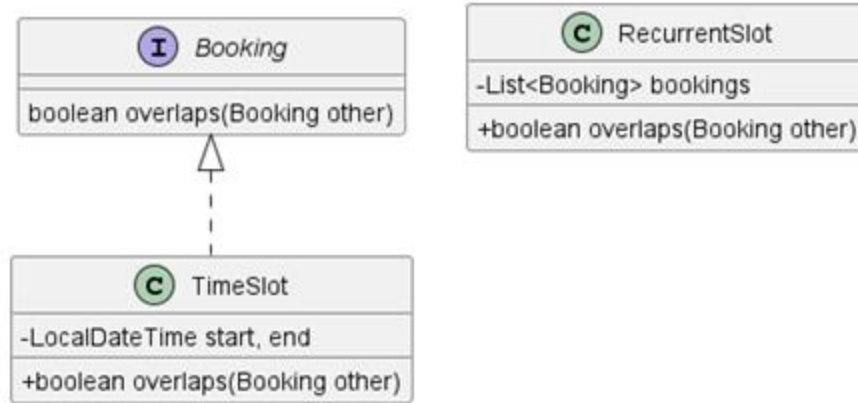
```
class SeniorEmployee extends Employee{
    ...
    public void addEmployee(Employee e) {
        subordinates.add(e);
    }
    public int getSubordinatesCount() {
        int count = subordinates.size();
        for (Employee e : subordinates) {
            count += e.getSubordinatesCount();
        }
        return count;
    }
}
```



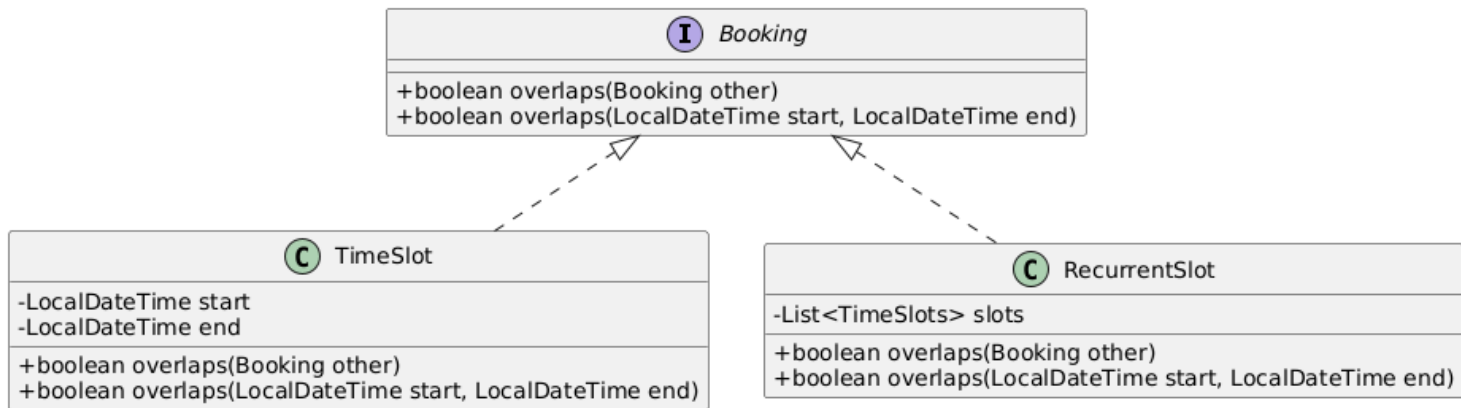
```
class JuniorEmployee extends Employee {
    ...
    public void addEmployee(Employee e) { }
    public int getSubordinatesCount() {
        return 0;
    }
}
```

Do RecurrentSlot and TimeSlot follow the Composite pattern?

No
Yes



Use the Composite pattern to implement TimeSlot and RecurrentSlot



```
interface Booking {
    boolean overlaps(Booking other);
    boolean overlaps(LocalDateTime start,
                     LocalDateTime end);
}
```

Use the Composite pattern to implement TimeSlot and RecurrentSlot

```
class TimeSlot implements Booking {
    private LocalDateTime start, end;

    public boolean overlaps(LocalTimeDate start,
                           LocalTimeDate end)
    {
        return (start<=this.end) && (end>=this.start)
               && start <= end
    }

    public boolean overlaps(Booking other) {
        return other.overlaps(this.start,this.end);
    }
}
```

```
class RecurrentSlot implements Booking {
    private List<Booking> slots;

    public boolean overlaps(LocalTime start,
                           LocalTime end) {
        for (TimeSlots s : slots) {
            res = res || s.overlaps(start,end);
        }
    }

    public boolean overlaps(Booking other) {
        boolean res = false;
        for (Booking s : slots) {
            res = res || other.overlaps(s);
        }
        return res;
    }
}
```

Template Method Pattern

- An algorithm that relies on some abstract methods.
- Subclasses provide implementation of some steps of the algorithm

```
public boolean removeAll(Collection<?> c) {  
    Objects.requireNonNull(c);  
    boolean modified = false;  
    if (size() > c.size()) {  
        for (Object e : c)  
            modified |= remove(e);  
    } else {  
        for (Iterator<?> i = iterator(); i.hasNext(); ) {  
            if (c.contains(i.next())) {  
                i.remove();  
                modified = true;  
            }  
        }  
    }  
    return modified;  
}
```

[java.util.AbstractSet](#)

Shape uses the template method pattern

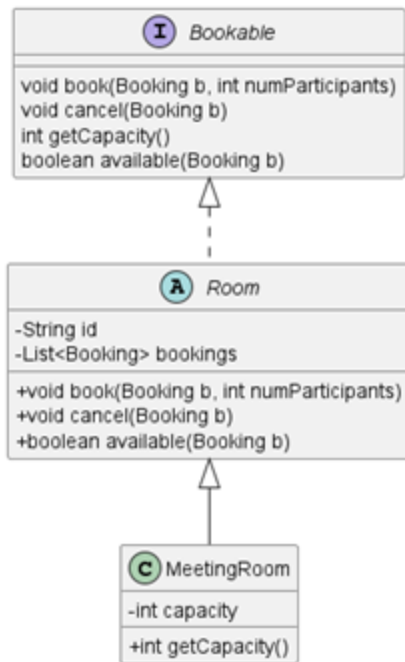
```
abstract class Shape {  
  
    private final double x;  
    private final double y;  
    ...  
    public abstract double distance(Shape other);  
  
    public abstract Point2D getClosestPoint(Point2D other);  
  
    public final boolean overlaps(Shape other) {  
        return distance(other) < 0;  
    }  
}
```

```
class Circle extends Shape {  
    private final double radius;  
    ...  
    public double distance(Shape s) { ... }  
  
    public Point2D getClosestPoint(Point2D other) { ... }  
}
```

Does Room use the template method pattern?

```
public void book(Booking b, int numParticipants) {  
    if (available(b) && getCapacity() <= numParticipants) {  
        bookings.add(b);  
    }  
}
```

```
public boolean available(Booking b) {  
    for (Booking b0 : bookings) {  
        if (b0.overlaps(b)) {  
            return false;  
        }  
    }  
    return true;  
}
```

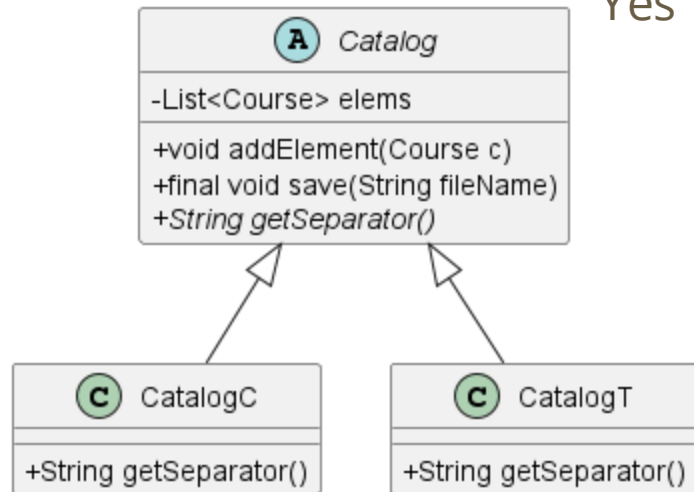


No
Yes

Does Catalog use the template method pattern?

No
Yes

```
abstract class Catalog {  
    private List<Course> elems;  
    ...  
  
    public final void save(String fileName) throws java  
        PrintWriter pw;  
        pw = new PrintWriter(fileName);  
        for (int i = 0; i < elems.size(); i++) {  
            Course c = elems.get(i);  
            pw.print(c);  
            if (i < elems.size() - 1) {  
                pw.print(getSeparator());  
            }  
        }  
        pw.print("\n");  
        pw.flush();  
        pw.close();  
    }  
    public abstract String getSeparator();  
}
```



Creational Patterns

- Simple Factory
- Factory Method
- Abstract Factory
- Builder
- ...

Structural Patterns

- Decorator
- Adapter
- Composite
- ...

Behavioral Patterns

- Template Method
- Observer
- Command
- Strategy
- Visitor
- ...

Appendix

Output

```
java.util.GregorianCalendar[time=1699484400000,areFieldsSet=true,areAllFieldsSet=true,
lenient=true,zone=sun.util.calendar.ZoneInfo[id="Europe/Copenhagen",offset=3600000,
dstSavings=3600000,useDaylight=true,transitions=143,lastRule=java.util.SimpleTimeZone[
id=Europe/Copenhagen,offset=3600000,dstSavings=3600000,useDaylight=true,startYear=0,
startMode=2,startMonth=2,startDay=-1,startDayOfWeek=1,startTime=3600000,
startTimeMode=2,endMode=2,endMonth=9,endDay=-1,endDayOfWeek=1,endTime=3600000,
endTimeMode=2]],firstDayOfWeek=2,minimalDaysInFirstWeek=4,ERA=1,YEAR=2023,
MONTH=10,WEEK_OF_YEAR=45,WEEK_OF_MONTH=2,DAY_OF_MONTH=9,DAY_OF_YEAR=313,
DAY_OF_WEEK=5,DAY_OF_WEEK_IN_MONTH=2,AM_PM=0,HOUR=0,HOUR_OF_DAY=0,MINUTE=0,
SECOND=0,MILLISECOND=0,ZONE_OFFSET=3600000,DST_OFFSET=0]
```

Thu Nov 09 00:00:00 CET 2023